El patrón Modelo-Vista-Controlador en JavaFX

Guía de estudio: qué es el MVC, para qué sirve, cómo se aplica y el ejemplo del contador explicado línea a línea.

PARTE 1 · EL PATRÓN

1. El problema: una clase que lo hace todo
2. Qué es el MVC y qué hace cada parte
3. Quién conoce a quién
4. El flujo de una acción y las dos formas de avisar a la vista
5. Ventajas, inconvenientes y el MVC en otras tecnologías

PARTE 2 · EL EJEMPLO DEL CONTADOR

6. Antes de empezar: cómo es una aplicación JavaFX por dentro
7. Qué hace el programa y cómo están organizados los ficheros
8. El modelo: `Contador`
9. La vista: `VistaContador`
10. El controlador: `ControladorContador`
11. El arranque: `App` y `Main`
12. Qué ocurre exactamente al pulsar «+»
13. El mismo contador con FXML
14. Para comprobar que lo has entendido

Código del ejemplo: proyecto `mvc-javafx`, paquetes `mvc.contador` y `mvc.contadorfxml`.

1 El problema: una clase que lo hace todo

Imagina una calculadora muy sencilla: el usuario escribe dos números, pulsa **+** y ve el resultado. Aunque sea un programa pequeño, su código tiene que hacer **tres cosas distintas**:

1. **Dibujar la ventana**: los campos de texto, el botón y la etiqueta del resultado.
2. **Reaccionar** cuando el usuario pulsa el botón.
3. **Hacer la cuenta**: la lógica de la suma.

Lo primero que se le ocurre a casi todo el mundo es meter esas tres cosas en la misma clase:

```
public class Calculadora extends Application {
    public void start(Stage ventana) {
        TextField a = new TextField();        // 1. ventana
        TextField b = new TextField();        // 1. ventana
        Label res = new Label();              // 1. ventana
        Button sumar = new Button("+");       // 1. ventana

        sumar.setOnAction(e -> {              // 2. reacción
            int x = Integer.parseInt(a.getText());
            int y = Integer.parseInt(b.getText());
            res.setText("" + (x + y));         // 3. lógica
        });
        // ... colocar los nodos y mostrar la ventana
    }
}
```

Funciona, pero tiene consecuencias que se notan en cuanto el programa crece:

- **No se puede probar la lógica sin abrir la ventana.** Para comprobar que la suma está bien hay que ejecutar el programa y pulsar botones a mano. No hay forma de escribir un test automático (JUnit) de `x + y`, porque la suma está metida dentro del `setOnAction` de un botón.
- **No se puede cambiar la ventana sin arriesgar la lógica.** Si mañana la interfaz pasa a diseñarse con FXML o a ser una página web, hay que reescribirlo todo, incluida la suma, porque está mezclada con `TextField` y `Label`.
- **Los datos viven dentro de los componentes gráficos.** El número está en el texto de un `TextField`. La interfaz hace de «almacén», y eso lleva a errores difíciles de encontrar.
- **Todo cambia a la vez.** Cualquier modificación toca la misma clase, y en un equipo varias personas acaban editando el mismo fichero.

> **Para verlo en grande**
>
> En el proyecto tienes `mvc.calculadoraespagueti.CalculadoraEspagueti`: una calculadora completa escrita así, con los «malos olores» marcados con comentarios `[OLOR]`. Verás un método `pulsar` enorme que lee la pantalla, calcula, valida, abre diálogos de error y pinta el resultado, todo a la vez. Y tiene un fallo escondido que cuesta encontrar precisamente por estar todo mezclado.

A este código en el que todo está enredado se le llama coloquialmente **código espagueti**. El MVC es la solución clásica a este problema.

2 Qué es el MVC y qué hace cada parte

El **Modelo-Vista-Controlador (MVC)** es un **patrón de arquitectura**: una forma probada de organizar el código de una aplicación con interfaz gráfica. La idea es sencilla: **separar el programa en tres partes y que cada una tenga una sola responsabilidad**.

Modelo

Los **datos** y las **reglas** de la aplicación.

¿Qué información hay y qué reglas se cumplen?

Vista

Lo que el usuario **ve y toca**: paneles, botones, etiquetas.

¿Cómo se muestran los datos?

Controlador

El **intermediario** entre los otros dos.

¿Qué pasa cuando el usuario actúa?

Qué hace y, sobre todo, qué NO hace cada parte

La parte de «qué NO hace» es la más importante: es lo que mantiene el código separado.

Parte	Qué hace	Qué NO hace	En JavaFX
Modelo	Guarda el estado (los datos), aplica las reglas y, si hace falta, avisa de que algo ha cambiado.	No dibuja nada ni conoce la vista. No importa `javafx.scene` ni `javafx.stage`.	Clases normales de Java. Si tiene que avisar de cambios: propiedades y `ObservableList`.
Vista	Pinta los datos que le pasan y avisa de las acciones del usuario (clics, teclas…).	No calcula ni decide. No tiene `if` sobre la lógica del programa.	Un panel (`BorderPane`, `VBox` …) con `Label`, `Button` … o un fichero FXML.
Controlador	Recibe las acciones de la vista, llama al modelo y actualiza la vista.	No guarda datos ni dibuja.	Una clase que se engancha a los botones (`setOnAction`) o con métodos `@FXML`.

> **Una analogía: el restaurante**
>
> La **cocina** es el modelo: ahí están los ingredientes (datos) y las recetas (reglas). La **sala y la carta** son la vista: es lo que ve el cliente. El **camarero** es el controlador: recoge lo que pide el cliente, se lo lleva a la cocina y vuelve con el plato. El cliente nunca entra en la cocina, y el cocinero no sale a la sala a tomar nota.

Cómo saber a qué parte pertenece un trozo de código

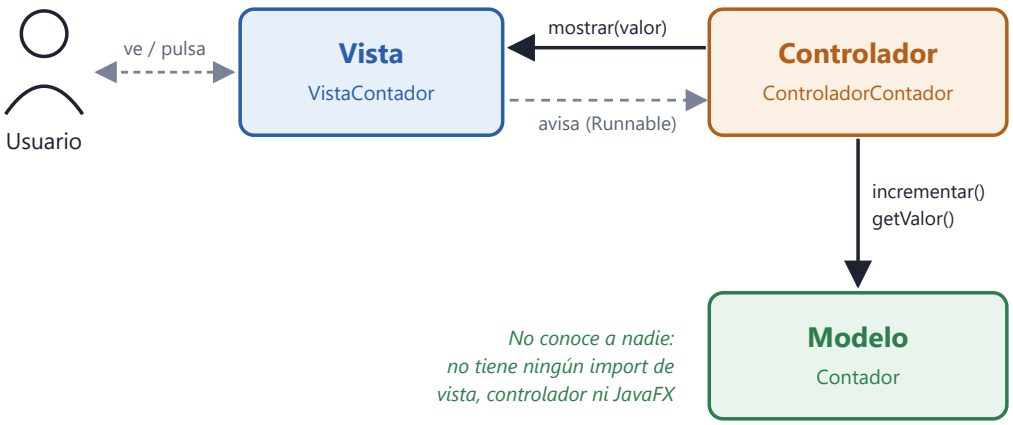
Ante cualquier línea de código, hazte estas preguntas:

- ¿Seguiría teniendo sentido si el programa fuera de consola o una web? → Es **modelo**. (La suma, el saldo de una cuenta, comprobar si una contraseña es correcta…)

- ¿Habla de colores, tamaños, posiciones, componentes o textos en pantalla? → Es **vista**.
- ¿Es del tipo «cuando pase X, haz Y»? → Es **controlador**.

3 Quién conoce a quién

Separar el código en tres clases no basta: también hay que fijar **qué clase puede usar a cuál**. En Java esto se ve en los `import`: si una clase importa otra, la conoce y depende de ella.



Flecha continua: «llama a métodos de». Flecha discontinua: la vista avisa ejecutando la acción que le ha dado el controlador, sin saber quién es.

> **Las reglas de dependencia**
>
> 1. El **controlador** conoce al modelo y a la vista. Es el único que importa a los otros dos.
> 2. La **vista** no conoce al modelo ni al controlador. Solo recibe datos ya preparados (por ejemplo, un `int`) y ofrece métodos para que alguien se suscriba a sus botones.
> 3. El **modelo** no conoce a nadie. No sabe que existe una ventana: nunca importa `javafx.scene` ni `javafx.stage`.

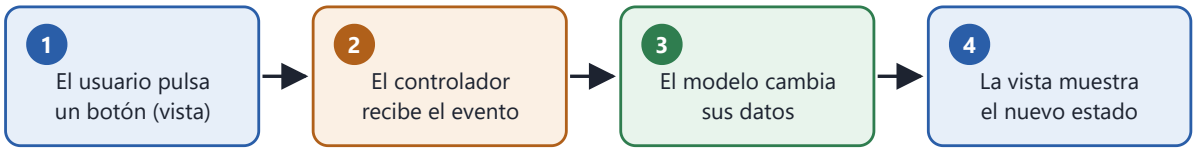
¿Por qué el modelo no puede conocer a nadie? Porque es la parte más valiosa del programa: contiene las reglas. Si no depende de la interfaz, se puede reutilizar tal cual en una aplicación de consola, en una web o con otra vista distinta, y se puede probar con JUnit sin abrir ninguna ventana. En cuanto el modelo importa un `Label` o un `Stage`, pierdes todo eso.

> **Un matiz de JavaFX**
>
> JavaFX trae **propiedades** y **listas observables** (paquetes `javafx.beans` y `javafx.collections`) que avisan solas cuando cambian. No dibujan nada y funcionan sin ventanas, así que el modelo **sí** puede usarlas. Lo verás en los ejemplos de la lista de la compra y de la tabla. El contador no las necesita: su modelo no importa nada.

¿Por qué la vista no conoce al modelo? Para poder cambiarla sin tocar nada más. La vista solo sabe «enseñar un número» y «avisar de que han pulsado un botón». Le da igual de dónde salga ese número.

4 El flujo de una acción

En una aplicación MVC, cada vez que el usuario hace algo se recorre siempre el mismo camino:



Fíjate en que la vista aparece al principio y al final, pero **nunca habla directamente con el modelo**: siempre pasa por el controlador.

Cómo se entera la vista de los cambios: dos variantes

El paso 4 se puede resolver de dos maneras. Las dos son MVC; la diferencia es **quién avisa a la vista**.

A · El controlador refresca

Después de cambiar el modelo, el controlador le pide el nuevo valor y se lo pasa a la vista:

```
modelo.incrementar();
vista.mostrar(modelo.getValor());
```

Más simple. Es ideal para empezar y es la que usa **el contador**.

B · El modelo avisa (patrón Observer)

El modelo guarda sus datos en propiedades observables y la vista se **enlaza** a ellas con `bind()`:

```
lblTotal.textProperty().bind(
    total.asString("Total: %.2f €"));
```

Varias vistas sincronizadas. La usan la *lista de la compra* y la *tabla de alumnos*.

`bind()` significa «el texto de esta etiqueta es **siempre** el total con este formato». Cuando el total cambia, el texto cambia solo. La variante B es útil cuando hay varias vistas que muestran los mismos datos: con la variante A, el controlador tendría que acordarse de refrescarlas todas; con la B, cada vista se enlaza una vez y ya no hay que hacer nada más.

5 Ventajas, inconvenientes y el MVC en otras tecnologías

Ventajas

- Cambiar la interfaz sin tocar la lógica.
- Probar el modelo con JUnit, sin ventanas.
- Reutilizar el modelo con otra vista (por ejemplo, en código o en FXML).
- Trabajar en equipo por capas: cada persona en una parte.
- Saber dónde buscar: un fallo de cálculo está en el modelo; uno de aspecto, en la vista.

Inconvenientes

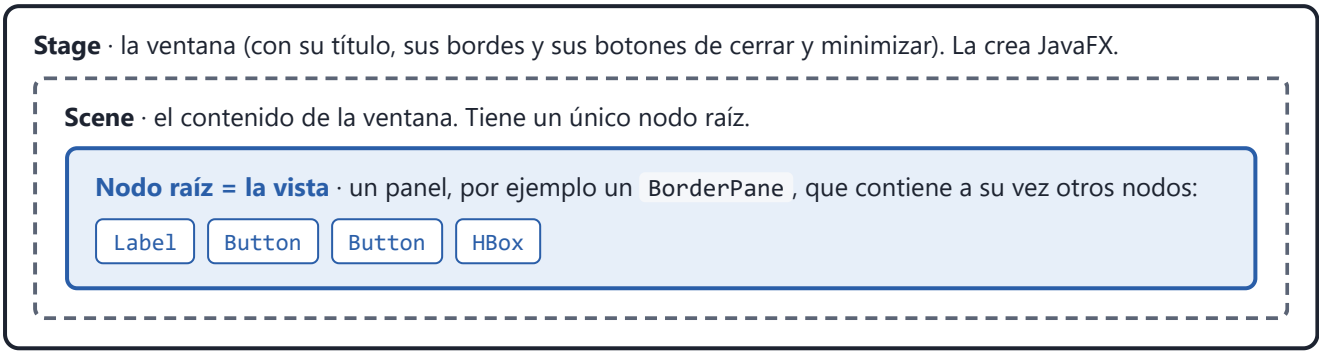
- Más clases y ficheros, incluso para apps pequeñas.
- Cuesta un poco al principio decidir qué va en cada sitio.
- Riesgo de controladores enormes si se mete en ellos lógica que debería estar en el modelo.

El MVC no es algo exclusivo de JavaFX. Lo vas a encontrar, con pequeñas variaciones, en casi todas las tecnologías con interfaz:

Tecnología	Modelo	Vista	Controlador
JavaFX (este curso)	Clases con propiedades y `ObservableList`	Panel en código o fichero FXML	Clase con `setOnAction` o `@FXML`
Swing	Clases Java, `TableModel`	`JFrame`, `JTable`	`ActionListener`
Spring MVC (web)	Entidades y servicios	Plantillas HTML	Clase `@Controller`
Android (MVVM)	Datos y repositorios	Layout XML	ViewModel

6 Antes de empezar: cómo es una aplicación JavaFX por dentro

Para entender dónde va cada pieza del contador, primero hay que conocer tres conceptos de JavaFX que aparecen en todos los ejemplos:



- **Stage:** la ventana. No la creamos nosotros: JavaFX la crea y nos la entrega ya hecha.
- **Scene:** lo que hay dentro de la ventana. Se crea con `new Scene(nodoRaiz)` y se coloca en la ventana con `ventana.setScene(...)`.
- **Nodos:** todo lo que se ve (`Label`, `Button`…) y los paneles que los colocan (`BorderPane`, `HBox`, `VBox`…). Se organizan en árbol: un panel contiene nodos, que pueden ser otros paneles.

Por eso, en el MVC con JavaFX, **la vista no es la ventana**: es un panel que se coloca dentro de ella.

El arranque: `Application`, `start()` y el hilo de JavaFX

Toda aplicación JavaFX tiene una clase que hereda de `Application` y escribe el método `start(Stage ventana)`. Cuando arranca el programa, JavaFX:

1. Prepara su **hilo de interfaz** (*JavaFX Application Thread*). Toda la interfaz se crea y se modifica desde ese hilo, y es también el que atiende los clics.
2. Crea la ventana principal (`Stage`).
3. Llama a `start(ventana)` desde ese hilo, entregándole la ventana. Ahí es donde creamos el modelo, la vista y el controlador.

> **¿Por qué cada ejemplo tiene un `Main` y un `App`?**
>
> Si la clase que tiene el `main()` hereda de `Application` y la ejecutas desde el IDE sin configurar nada, Java se niega a arrancar con el error `JavaFX runtime components are missing`. Con un JDK en español el mensaje sale mal traducido (*«el método principal debe devolver un valor del tipo void en la clase {0}»*), pero es el mismo problema.
>
> La solución habitual es separar las dos cosas: `App` hereda de `Application` y monta la aplicación, y `Main` es una clase normal cuyo `main()` solo llama a `Application.launch(App.class)`. **Ejecuta siempre `Main`, nunca `App`.**

7 El ejemplo del contador

Es la aplicación MVC más pequeña posible: una ventana con un número grande y dos botones. **«+»** suma uno al número y **«Reset»** lo vuelve a poner a cero.



La ventana del contador después de pulsar «+» tres veces.

Con tan poca funcionalidad, el código podría caber en diez líneas dentro de una sola clase. Precisamente por eso es un buen ejemplo: como el «qué hace» es trivial, puedes centrarte solo en el «**dónde va cada cosa y por qué**».

Organización de los ficheros

Cada parte del patrón va en su propio **paquete**. Así, con solo mirar el árbol del proyecto, cualquiera sabe dónde buscar:

```
mvc-javafx/src/main/java/mvc/contador/
├── Main.java                       ← el punto de entrada: solo lanza App
├── App.java                        ← crea las tres piezas, las conecta y abre la ventana
├── modelo/
│   └── Contador.java               ← el dato (valor) y las operaciones
├── vista/
│   └── VistaContador.java          ← el panel: etiqueta y botones
└── controlador/
    └── ControladorContador.java    ← une vista y modelo
```

Clase	Parte	Responsabilidad	Importa…
`Contador`	**Modelo**	Guardar el número y saber incrementarlo y reiniciarlo.	Nada.
`VistaContador`	**Vista**	Dibujar el panel, mostrar un número y avisar de los clics.	Solo JavaFX.
`ControladorContador`	**Controlador**	Cuando se pulsa un botón, cambiar el modelo y refrescar la vista.	`Contador` y `VistaContador`.
`App`	Arranque	Crear las tres piezas, conectarlas y mostrar la ventana.	Las tres y JavaFX.
`Main`	Arranque	Lanzar la aplicación con `Application.launch`.	Solo `Application`.

Fíjate en la última columna: coincide exactamente con las reglas de dependencia del apartado 3. Comprobar los `import` de cada clase es la forma más rápida de saber si un proyecto respeta el MVC.

8 El modelo: `Contador` MODELO

`modelo/Contador.java`

```
package mvc.contador.modelo;                          ①

public class Contador {                               ②

    private int valor;                                ③

    public void incrementar() {                       ④
        valor++;
    }

    public void reiniciar() {
        valor = 0;
    }

    public int getValor() {                           ⑤
        return valor;
    }
}
```

① **Paquete `modelo`.** Agrupa todo lo que es «el dominio» del programa. En aplicaciones más grandes aquí habría varias clases (por ejemplo, `Producto` y `ListaCompra`).

② **Una clase Java normal.** No hereda de nada ni tiene **ningún `import`**, ni siquiera de JavaFX. Es lo más importante del modelo: no sabe que existe una ventana. Por eso esta misma clase serviría sin cambios en un programa de consola, en una web o en un test. De hecho, el ejemplo `mvc.contadorfxml` la usa tal cual con una vista completamente distinta (apartado 13).

③ **El estado es `private`.** Nadie desde fuera puede hacer `contador.valor = -50`. La única forma de cambiar el valor es a través de los métodos del modelo, así que **el modelo controla sus propias reglas**. Un `int` se inicializa a 0 automáticamente, por eso no hace falta constructor.

④ **Métodos con nombre de acción, no *setters*.** En lugar de un `setValor(int)` genérico, el modelo ofrece exactamente las operaciones que tienen sentido: `incrementar()` y `reiniciar()`. Si mañana se añade una regla (por ejemplo, «el contador no pasa de 99»), se escribe aquí, en un único sitio, y todas las vistas la respetan sin tocarlas.

⑤ **Un *getter* para leer el estado.** Es lo que usará el controlador para saber qué número debe enseñar la vista.

La prueba de que el modelo es independiente

Como el modelo no depende de JavaFX, se puede probar sin abrir ninguna ventana. Este test de JUnit 5 comprueba sus reglas en milisegundos:

```
class ContadorTest {
    @Test
    void incrementarSumaUno() {
        Contador c = new Contador();
        c.incrementar();
        c.incrementar();
        assertEquals(2, c.getValor());
    }

    @Test
    void reiniciarVuelveACero() {
        Contador c = new Contador();
        c.incrementar();
        c.reiniciar();
        assertEquals(0, c.getValor());
    }
}
```

Con el código espagueti del apartado 1, esto sería imposible: para probar la lógica habría que abrir la ventana y pulsar los botones a mano. (Para ejecutar este test hay que añadir JUnit al `pom.xml`, como en el proyecto `soluciones-javafx`.)

9 La vista: `VistaContador` VISTA

vista/VistaContador.java

```
package mvc.contador.vista;

import javafx.geometry.Insets;                                          ①
import javafx.geometry.Pos;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;

public class VistaContador extends BorderPane {                        ②

    private final Label lblValor = new Label("0");                      ③
    private final Button btnIncrementar = new Button("+");
    private final Button btnReset = new Button("Reset");

    public VistaContador() {                                            ④
        lblValor.setFont(Font.font("System", FontWeight.BOLD, 48));

        HBox panelBotones = new HBox(10, btnIncrementar, btnReset);
        panelBotones.setAlignment(Pos.CENTER);
        panelBotones.setPadding(new Insets(10));

        setCenter(lblValor);       // el centro de un BorderPane se centra solo
        setBottom(panelBotones);
    }

    public void mostrar(int valor) {                                    ⑤
        lblValor.setText(String.valueOf(valor));
    }

    public void alPulsarIncrementar(Runnable accion) {                  ⑥
        btnIncrementar.setOnAction(e -> accion.run());
    }

    public void alPulsarReset(Runnable accion) {
        btnReset.setOnAction(e -> accion.run());
    }
}
```

En el fichero original hay además comentarios *javadoc*; aquí se han quitado porque repiten lo que se explica a continuación.

Explicación de la vista

① **Solo importa JavaFX.** No hay ningún `import mvc.contador.modelo…` ni `…controlador…`. La vista no sabe qué es un `Contador` ni quién la va a controlar.

② **Hereda de `BorderPane`.** La vista es un **panel**, no la ventana: la ventana (`Stage`) la crea JavaFX y la vista se coloca dentro de ella (apartado 6). Un `BorderPane` divide su espacio en cinco zonas: arriba, abajo, izquierda, derecha y centro. Aquí solo usamos el centro (el número) y abajo (los botones).

③ **Los componentes son `private final`.**

- `private`: nadie de fuera puede tocar directamente la etiqueta ni los botones. Si el controlador pudiera hacer `vista.lblValor.setText(...)`, estaría dependiendo de que la vista use un `Label`; cambiarlo por otro componente rompería el controlador.
- `final`: se crean una sola vez y no se sustituyen nunca.

④ **El constructor solo monta el panel.** Pone una fuente grande y en negrita a la etiqueta, mete los dos botones en un `HBox` (una fila, con 10 píxeles de separación, centrada y con un margen interior de 10 píxeles) y coloca la etiqueta en el centro y la fila de botones abajo. Todo es aspecto: tamaños, posiciones, fuentes. **No hay ni un `if` ni un cálculo.**

⑤ **`mostrar(int valor)`: la vista solo pinta lo que le dan.** Recibe un número ya calculado y lo pone en la etiqueta. No sabe de dónde viene ni por qué es ese. Fíjate en que recibe un `int`, no un `Contador`: así sigue sin depender del modelo.

⑥ **Los «enganches» `alPulsarX(Runnable)`.** Es la pieza más interesante. La vista no decide qué pasa al pulsar un botón, pero tiene que dejar que *alguien* lo decida. Para eso ofrece un método que recibe «lo que hay que hacer» y lo engancha al botón:

- Un `Runnable` es simplemente «algo que se puede ejecutar» con `run()`. El controlador le pasará una lambda como `() -> incrementar()`.
- `setOnAction(...)` es la forma de JavaFX de decir «cuando pulsen este botón, haz esto». Recibe una lambda con el evento como parámetro (`e`), que aquí no se usa: simplemente se ejecuta la acción con `accion.run()`.
- ¿Por qué no pasarle al controlador el botón, o pedirle directamente un manejador de eventos de JavaFX? Porque así **el controlador no necesita importar nada de JavaFX** ni saber que hay un `Button`. Si mañana el «+» pasa a ser una opción de menú o un atajo de teclado, solo cambia la vista.

> **Ojo: `setOnAction` sustituye, no suma**
>
> Un botón de JavaFX solo tiene **una** acción. Si llamas dos veces a `alPulsarIncrementar`, la segunda acción reemplaza a la primera. En este ejemplo no es un problema, porque solo hay un controlador.

> **Comprueba tu vista con esta pregunta**
>
> ¿Podrías cambiar la etiqueta por un `TextField`, poner el número en rojo o mover los botones arriba **sin tocar ningún otro fichero**? Si la respuesta es sí, la vista está bien separada.

10 El controlador: `ControladorContador` CONTROLADOR

controlador/ControladorContador.java

```
package mvc.contador.controlador;

import mvc.contador.modelo.Contador;                    ①
import mvc.contador.vista.VistaContador;

public class ControladorContador {

    private final Contador modelo;                      ②
    private final VistaContador vista;

    public ControladorContador(Contador modelo, VistaContador vista) {    ③
        this.modelo = modelo;
        this.vista = vista;

        // Nos suscribimos a los botones de la vista
        vista.alPulsarIncrementar(() -> incrementar());    ④
        vista.alPulsarReset(() -> reiniciar());

        refrescarVista(); // mostrar el estado inicial     ⑤
    }

    private void incrementar() {                        ⑥
        modelo.incrementar(); // 1) cambiar el modelo
        refrescarVista();     // 2) actualizar la vista
    }

    private void reiniciar() {
        modelo.reiniciar();
        refrescarVista();
    }

    private void refrescarVista() {                     ⑦
        vista.mostrar(modelo.getValor());
    }
}
```

① **Es la única clase que importa a las otras dos.** Es lo esperado: el controlador es el «pegamento» y tiene que conocer a ambos. Fíjate en que **no importa nada de JavaFX**: gracias a los enganches con `Runnable`, no necesita saber que existen los botones.

② **Guarda una referencia al modelo y otra a la vista.** Las necesita para usarlas más tarde, cada vez que se pulse un botón. Son `final` porque un controlador trabaja siempre con el mismo modelo y la misma vista.

③ **Recibe el modelo y la vista por el constructor, no los crea.** Esto se llama *inyección de dependencias*. Quien crea las piezas es `App`. Así el controlador podría trabajar con un modelo que ya tuviera datos, o con otra vista compatible.

④ **Se suscribe a los botones usando los enganches de la vista.** `() -> incrementar()` es una *lambda* sin parámetros: un `Runnable` que, cuando se ejecute, llamará a `incrementar()`. Se lee así: «cuando

pulsen el botón de incrementar, llama a mi método `incrementar()`».

⑤ **Muestra el estado inicial.** Sin esta línea, la etiqueta enseñaría el `"0"` con el que se creó en la vista. Aquí coincide, pero si el modelo empezara en otro valor (por ejemplo, cargado de un fichero), la pantalla mentiría. La regla es: **lo que se ve siempre sale del modelo**.

⑥ **Cada acción sigue siempre los mismos dos pasos:** 1) pedir al modelo que cambie y 2) refrescar la vista. Observa lo que *no* hace: no hace `valor++` él mismo (eso es del modelo) ni toca ningún `Label` (eso es de la vista). Solo coordina.

⑦ **Un único método para refrescar.** Lee el estado del modelo y se lo pasa a la vista. Al tenerlo en un solo sitio, no se repite código y es imposible que un botón actualice la pantalla de una forma y otro de otra. Esta es la **variante A** del apartado 4: el controlador es quien refresca la vista.

Error típico: el controlador «gordo»

Si en el controlador empiezas a escribir cálculos o reglas («si el valor llega a 10, ponlo a 0»), estás metiendo lógica de modelo en el controlador. Esa regla debe ir en `Contador`, para que se cumpla siempre y se pueda probar con JUnit. El controlador debería ser corto y aburrido.

11 El arranque: `App` y `Main` ARRANQUE

App.java

```java
package mvc.contador;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import mvc.contador.controlador.ControladorContador;
import mvc.contador.modelo.Contador;
import mvc.contador.vista.VistaContador;

public class App extends Application {                          ①

    @Override
    public void start(Stage ventana) {                          ②
        Contador modelo = new Contador();                       ③
        VistaContador vista = new VistaContador();
        new ControladorContador(modelo, vista);                 ④

        // Stage = la ventana; Scene = su contenido; la vista es el nodo raíz de la escena
        ventana.setTitle("MVC - Contador");                     ⑤
        ventana.setScene(new Scene(vista, 300, 200));
        ventana.show();
    }
}
```

Main.java

```java
package mvc.contador;

import javafx.application.Application;

public class Main {                                             ⑥

    public static void main(String[] args) {
        Application.launch(App.class, args);
    }
}
```

① **`App` hereda de `Application`.** Es la aplicación JavaFX propiamente dicha. Su trabajo es **crear y conectar** las piezas; no contiene lógica.

② **`start(Stage ventana)`** lo llama JavaFX desde su hilo de interfaz y le entrega la ventana ya creada (apartado 6). Por eso aquí no hay que preocuparse de hilos: todo lo que se haga dentro de `start` ocurre en el hilo correcto.

③ **Se crean las piezas por separado.** Primero el modelo y la vista, que no se conocen entre sí.

④ **El controlador las conecta.** Al crearlo se ejecuta su constructor, que engancha los botones y muestra el valor inicial. No se guarda en ninguna variable porque no hace falta: los botones de la vista guardan las lambdas, y esas lambdas mantienen vivo al controlador.

⑤ **Se monta y se muestra la ventana al final**, cuando todo está ya conectado: título, escena (con la vista como nodo raíz y un tamaño de 300 × 200 píxeles) y `show()`. Así el usuario nunca ve una ventana con botones que todavía no hacen nada.

⑥ **`Main` no hereda de `Application`.** Su `main()` solo llama a `Application.launch(App.class, args)`, que arranca JavaFX y acaba llamando a `App.start()`. Existe para evitar el error `JavaFX runtime components are missing` al ejecutar desde el IDE (apartado 6). **Es la clase que hay que ejecutar.**

> **`App` es el único sitio donde «todo se conoce»**
>
> Si mañana quisieras dos ventanas que comparten el mismo contador, bastaría con cambiar `App`: crear un modelo, dos vistas y dos controladores que reciban el mismo `modelo`. (Ojo: con la variante A, una ventana no se enteraría de los cambios hechos desde la otra. Ese es justo el problema que resuelven las propiedades observables en el ejemplo de la lista de la compra.)

12 Qué ocurre exactamente al pulsar «+»

Juntando todo, este es el recorrido completo desde que arranca el programa hasta que el número cambia en pantalla.

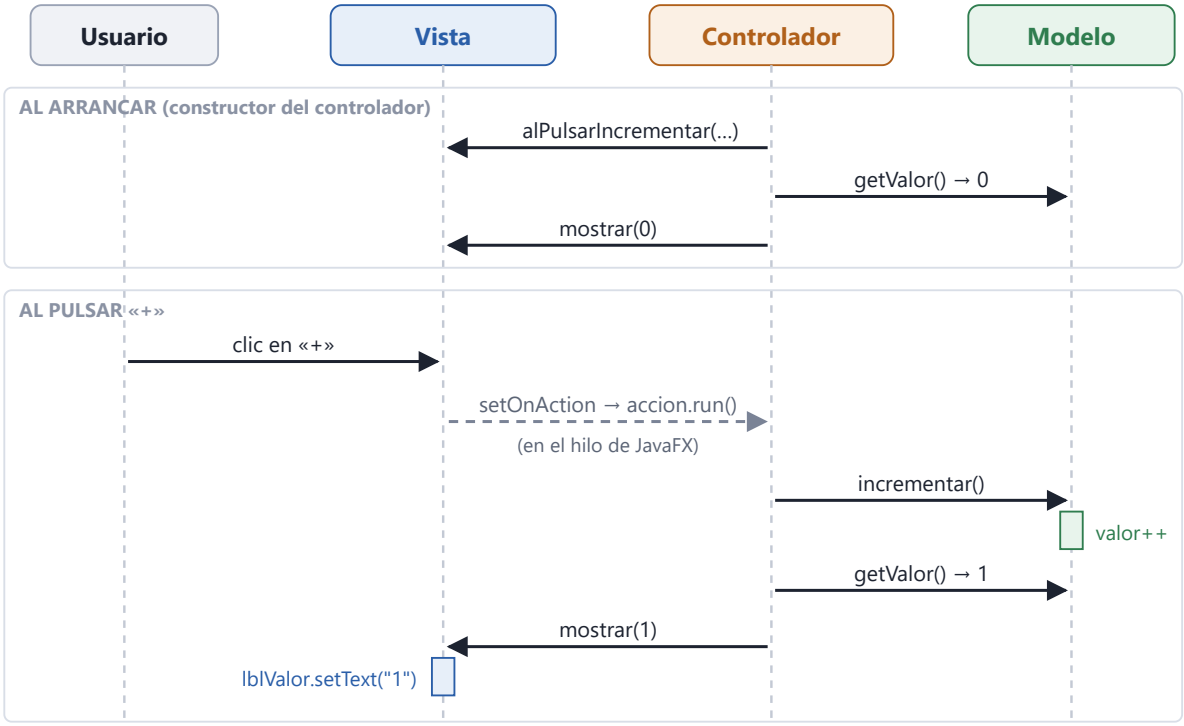


Diagrama de secuencia: cada flecha es una llamada a un método. Lee de arriba abajo.

1. **Arranque.** `Main` llama a `Application.launch`. JavaFX prepara su hilo, crea la ventana y llama a `App.start()`, que crea el modelo (`valor = 0`), la vista y el controlador.
2. **El controlador se engancha** a los botones con `alPulsarIncrementar` y `alPulsarReset`. Cada enganche hace `setOnAction(e -> accion.run())` sobre su botón. Después llama a `refrescarVista()`: la etiqueta muestra **0**. Por último, `App` muestra la ventana.
3. **El usuario pulsa «+».** JavaFX ejecuta, en su hilo, la acción que tenía guardada el botón: `e -> accion.run()`. Esa `accion` es la lambda `() -> incrementar()` del controlador.
4. **El controlador cambia el modelo:** `modelo.incrementar()` → dentro del modelo, `valor++`. Ahora vale 1.
5. **El controlador refresca la vista:** `vista.mostrar(modelo.getValor())`, es decir, `vista.mostrar(1)`.
6. **La vista pinta:** `lblValor.setText("1")`. El usuario ve el 1.

Con «Reset» el recorrido es idéntico, cambiando `incrementar()` por `reiniciar()`. Esta repetición es buena señal: todas las acciones siguen el mismo patrón y el código es predecible.

13 El mismo contador con FXML

En JavaFX es muy habitual escribir la vista en un fichero **FXML**: un XML que describe qué nodos hay y cómo se colocan, y que se puede editar visualmente con **Scene Builder**. El paquete `mvc.contadorfxml` es el mismo contador hecho así.

src/main/resources/mvc/contadorfxml/vista/VistaContador.fxml (resumido)

```
<BorderPane fx:controller="mvc.contadorfxml.controlador.ControladorContador"
            prefWidth="300" prefHeight="200">
    <center>
        <Label fx:id="lblValor" text="0"> ... </Label>
    </center>
    <bottom>
        <HBox alignment="CENTER" spacing="10">
            <Button text="+" onAction="#incrementar"/>
            <Button text="Reset" onAction="#reiniciar"/>
        </HBox>
    </bottom>
</BorderPane>
```

controlador/ControladorContador.java (paquete mvc.contadorfxml)

```
public class ControladorContador {

    private final Contador modelo;

    @FXML
    private Label lblValor; // lo rellena el FXMLLoader gracias a fx:id="lblValor"

    public ControladorContador(Contador modelo) {
        this.modelo = modelo;
    }

    @FXML
    private void initialize() {   // se ejecuta cuando ya se han inyectado los @FXML
        refrescarVista();
    }

    @FXML
    private void incrementar() {  // onAction="#incrementar" en el FXML
        modelo.incrementar();
        refrescarVista();
    }
    // ... reiniciar() y refrescarVista() igual que antes
}
```

Qué cambia respecto a la versión en código

- **No hay clase `VistaContador`:** la vista es el fichero FXML. No tiene ni una línea de lógica.
- `fx:id="lblValor"` hace que el `FXMLLoader` meta ese `Label` en el atributo `@FXML lblValor` del controlador. Por eso **en el constructor `lblValor` todavía es `null`**: hay que esperar a `initialize()`.

- `onAction="#incrementar"` conecta el botón directamente con el método `@FXML incrementar()`. Ya no hacen falta los enganches `alPulsarX`.
- **El modelo es exactamente el mismo:** `App` importa `mvc.contador.modelo.Contador`, el del apartado 8, sin cambiar ni una línea. Es la ventaja del apartado 3 hecha realidad.
- **La fábrica de controladores.** El FXML dice qué clase es el controlador, pero el `FXMLLoader` no sabe crearla con el modelo. Por eso `App` le dice cómo: `cargador.setControllerFactory(tipo -> new ControladorContador(modelo));`

> **Para debatir**
>
> En la versión FXML, el controlador sí conoce un `Label` y lo modifica él mismo, algo que en la versión en código evitábamos. Es la forma habitual de trabajar con JavaFX, pero mezcla un poco controlador y vista. ¿Qué se gana y qué se pierde respecto a `mvc.contador`?

14 Para comprobar que lo has entendido

Lista de comprobación para tus propios proyectos MVC

- ☐ Hay tres paquetes: `modelo`, `vista` y `controlador`, más un `App` y un `Main`.
- ☐ Ninguna clase del modelo importa `javafx.scene` ni `javafx.stage`.
- ☐ La vista no importa nada del modelo ni del controlador.
- ☐ Los componentes de la vista son `private`, y la vista ofrece métodos `mostrar…(…)` y `alPulsar…(Runnable)`.
- ☐ La vista no contiene ningún cálculo ni ningún `if` sobre la lógica del programa.
- ☐ Los atributos del modelo son `private` y solo cambian a través de sus métodos.
- ☐ El controlador no hace cálculos: llama al modelo y refresca la vista.
- ☐ Lo que se ve en pantalla sale siempre del modelo (también al arrancar).
- ☐ `Main` no hereda de `Application` y es la clase que se ejecuta.
- ☐ Hay al menos un test JUnit del modelo, que se ejecuta sin abrir ventanas.

Preguntas

1. ¿En qué clase o clases tendrías que hacer cambios para añadir un botón **«−»** que reste uno? Escribe los métodos nuevos de cada una.
2. Haz lo mismo en la versión FXML. ¿Qué ficheros cambian ahora?
3. Te piden que el contador **no pueda pasar de 10**. ¿Dónde pondrías esa regla y por qué no en el controlador? Escribe un test JUnit que la compruebe.
4. Quieres que el número se vea **en rojo cuando sea mayor que 5**. ¿Es una regla del modelo o algo de la vista? Hay más de una respuesta razonable: argumenta la tuya.
5. Si quitas la llamada a `refrescarVista()` del constructor del controlador, ¿qué cambia en este programa? ¿Y si el modelo empezara en 7?
6. ¿Qué tendrías que cambiar para convertir el contador en una aplicación de **consola** (escribir «+» o «r» por teclado)? ¿Qué clase se reutiliza sin tocarla?
7. Abre `CalculadoraEspagueti`. Para cada trozo de su método `pulsar`, indica si pertenece al modelo, a la vista o al controlador.

> **Siguientes ejemplos del proyecto**
>
> **Login** (`mvc.login`): una regla de negocio de verdad en el modelo (`UsuarioService.validar`). **Lista de la compra** (`mvc.compra`): la variante B, con dos vistas que se actualizan solas gracias a `ObservableList` y `bind()`. **Tabla de alumnos** (`mvc.tableview`): una `TableView` que observa la lista del modelo.